Computational Data Analysis (02582)

Subspace models

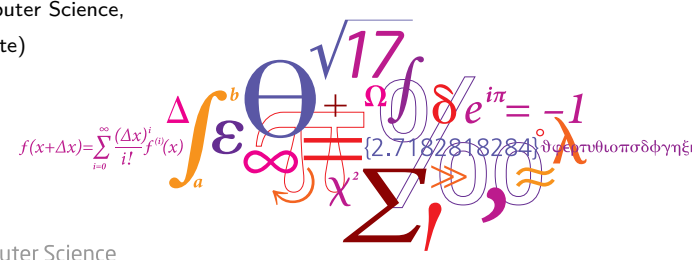
Sneha Das

Assistant Professor

Department of Applied Mathematics and Computer Science,

Technical University of Denmark (DTU Compute)

(Slides adapted from Line Clemmensen)



Outline I

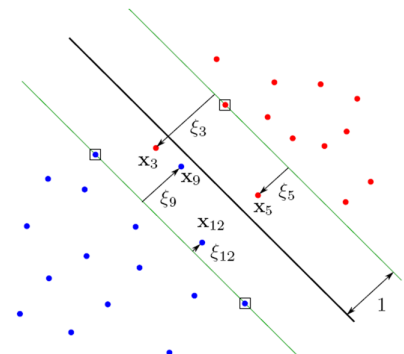
- Principal Component Analysis
- Sparse PCA
- Partial Least squares
- Canonical Correlation analysis
- APPENDIX

Last Week, The support Vector Machine

A two category classifier,

- Points that touch the boundary of the margin are support points (marked with squares)
- Budget for overlap (introduce slack variables)

$$\begin{cases} \arg\min_{\beta,\beta_0} \frac{1}{2}||\beta||^2 + \lambda \sum_{i=1}^{n} \xi_i \\ \text{such that} \\ \quad y_i(x_i\beta - \beta_0) \geq 1 - \xi_i \quad \forall i \\ \quad \xi_i \geq 0 \quad \forall i \end{cases}$$

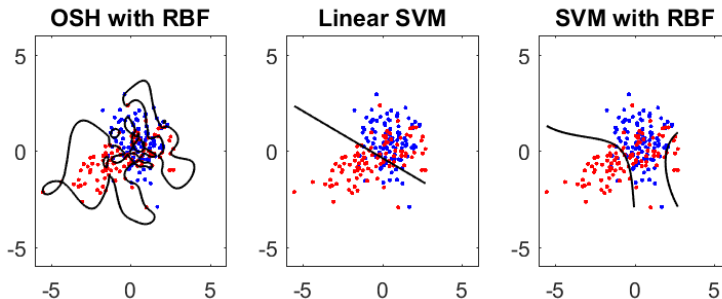


Last Week, The kernel trick

Replace XX^T with a basis expansion $h(X)h(X)^T$. The kernel trick is to *implicitly* define the basis expansion $K(X) = h(X)h(X)^T$ through a kernel function.
Most common choice is the Radial Basis Function,

$$K_{i,j} = \exp\left(-\frac{1}{c}||x_i - x_j||^2\right)$$

Last Week, Tuning the SVM



Separation can be obtained with

- non-linear boundaries,
- allowing for overlap,

or optimally by using both.

The Dimensionality Problem: Curse of High-Sample Data

- In modern applications like Speech Emotion Recognition (SER), data is highly dimensional.
- Example: A 2-second audio clip yields thousands of Mel-spectrogram features.
- We often face $p \gg n$ (More features than observations).
- **How do we find the meaningful subspace?**
  - *Unsupervised compression:* PCA / SVD
  - *Interpretable extraction:* Sparse PCA
  - *Supervised projection:* Partial Least Squares (PLS)
  - *Multi-modal alignment:* Canonical Correlation Analysis (CCA)

Principal Component Analysis

Regression and classification are confirmatory

- **Answers** to particular questions
  - Does wine-drinking influence heart disease? (regression)
  - How well can we separate between normal and abnormal ECG? (classification)
- **Supervised** - solutions are governed by the outcome variable

PCA is exploratory

- **Explore** examples of typical (common) observations based on your data set.
- **Unsupervised**, no outcome variable - let data speak for itself.
  - Structure in data
  - Outlier detection
  - Dimensionality reduction (data compression)

PCA and the Curse of Dimensionality

The Volume Explosion:

- In 1-D: 100 points occupy 75% of the space.
- In 10-D: You need 100^{10} points to get the same coverage.

The Manifold Hypothesis:

High-dimensional data (images, audio) is almost never truly p-dimensional. It lives on a low-dimensional manifold.
Why can we still learn from it?

- **Clustering:** Data is naturally grouped.
- **Correlation:** Features are redundant.
- **Structure:** Physical constraints limit variability.

PCA recovers this hidden structure.

Intuition

A high-res photo has 1,000,000 pixels (p).
If we change pixels randomly, we get 'noise'.
Real photos only occupy a tiny portion of that 1M-dimensional space.

Principal Component Analysis - The Idea

Linear Transformation:

$$S = XL$$

We want to preserve relations (angles) between variables, subject to $L^T L = I$.

- **Orthogonal transformation:** L is a rotation matrix.

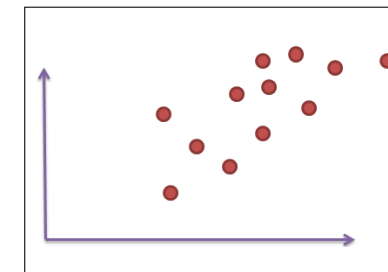
Which rotation?

- Rotate such that the projected data S has maximal variance.
- Successively maximize the variance of the principal components. Loading l_m solves:

$$\arg\max_{\alpha} Var(X\alpha)$$

subject to $||\alpha|| = 1,\ \alpha^T l_j = 0,\ j = 1, ..., m-1$

Example: 12 observations, 2 dimensions



Let Z be the raw observations. We want to find the axis through this 2D cloud that captures the widest possible spread of the data points.

PCA - Preprocessing & Coordinate Rotation

1. Center the Data

- Remove the mean from raw observations Z:

$$X = Z - \mu_Z$$

2. Rotate Coordinate System

- Project data onto the first axes in the direction of maximal variance:

$$S = XL$$

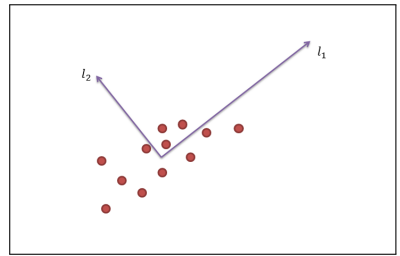
3. New Coordinates (Scores)

- Observations are now given as coordinates in a new coordinate system:

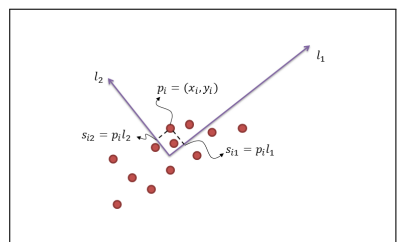
$$s_i = x_i L$$

$$s_i = (s_{i1}, s_{i2})$$

Visualizing the Transformation



Step 1&2: Centering to the origin and Rotating to capture maximal variance



Step 3: New coordinate

PCA Derivation (Step 1): Formulating the Objective

1. The Transformation:

$$S = XL, \quad \text{where } L^T L = I$$

2. Maximize Variance:

- Maximize variance of projected data $\implies$ maximize variance of each PC (columns of S):

$$\text{cov}(S) = \frac{1}{n} S^T S = \frac{1}{n} L^T X^T X L$$

$$\text{cov}(S) = L^T \Sigma L$$

where $\Sigma = \text{cov}(X)$

3. The First PC Objective:

$$\arg\max_{l} l^T \Sigma l \quad \text{s.t. } l^T l = 1$$

Numerical Example: Objective

Let $\Sigma = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$ (Variance is 3 on x-axis, 1 on y-axis).

Test Vector A (x-axis): $l_A = [1, 0]^T$

$$l_A^T \Sigma l_A = [1, 0] \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \mathbf{3}$$

Test Vector B (y-axis): $l_B = [0, 1]^T$

$$l_B^T \Sigma l_B = [0, 1] \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \mathbf{1}$$

Conclusion: l_A gives a higher variance. The math aims to find this optimal vector analytically.

PCA Derivation (Step 2): The Lagrangian & Calculus

4. The Lagrangian Formulation:

- To solve the constrained optimization, we construct the Lagrangian L_p with multiplier λ:

$$L_p = l^T \Sigma l - \lambda(l^T l - 1)$$

5. Vector Calculus (Optimization):

- Take the partial derivative of L_p with respect to the vector l and set it to zero:

$$\frac{\partial L_p}{\partial l} = 2\Sigma l - 2\lambda l = 0$$

Note: The derivative of the quadratic form $l^T \Sigma l$ is $2\Sigma l$ because Σ is symmetric.

Why use a Lagrangian?

We want to maximize $l^T \Sigma l$. Without a constraint, the math could just make $l = [1000, 0]^T$, resulting in a variance of $3,000,000$.

The penalty term $-\lambda(l^T l - 1)$ acts as:

- If length $l^T l > 1$, the penalty heavily subtracts from the score.
- If length $l^T l = 1$, the penalty is 0, and we are purely maximizing the valid variance.

PCA Derivation (Step 3): The Eigenvalue Problem

6. The Eigenvalue Equation:

- Rearranging the result from our derivative ($2\Sigma l - 2\lambda l = 0$), we get:

$$\Sigma l = \lambda l$$

The Mathematical Conclusion:

- Covariance is maximized when l is an eigenvector of Σ.
- The maximal variance achieved is exactly its corresponding eigenvalue λ.

Remaining Components:

Because Σ is symmetric, its eigenvectors are automatically orthogonal. Thus, $l_2, l_3 \ldots$ naturally form uncorrelated PC axes.

Numerical Example: Eigenvalues

Recall our matrix $\Sigma = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$.

The equation $\Sigma l = \lambda l$ asks: *Which vector l only changes scale, not direction, when multiplied by Σ?*

Let's test our best vector from Step 1 ($l_A = [1, 0]^T$):

$$\begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \end{bmatrix} = \mathbf{3} \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Perfect match! $l = [1, 0]^T$ is the Eigenvector, and $\lambda = 3$ is the Eigenvalue (the exact maximum variance we calculated earlier).

PCA - Using Singular value Decomposition

Singular value decomposition of data matrix X: $X = US_dV^T$, where the left singular vectors are orthonormal $U^TU = I$, the right singular values are orthonormal $V^TV = I$, and S_d is a diagonal matrix with the singular values.
Note that

- $V = L$: The right singular vectors correspond to the eigenvectors, i.e. the loadings of the PCA
- $S_d = \sqrt{nD}$: The singular values are equivalent to the eigen values scaled. Recall, the eigenvalues expresses the variances corresponding to the principal components.

PCA Computation - The SVD Connection

1. Singular Value Decomposition (SVD)

- Decompose the centered data matrix X:

$$X = UDV^T$$

(U: Left singular vectors, D: Diagonal singular values, V: Right singular vectors)

2. Connection to Covariance (EVD)

- Recall $\Sigma \propto X^T X$. Substitute the SVD:

$$X^T X = (UDV^T)^T (UDV^T)$$

$$X^T X = VDU^T UDV^T$$

- Since U is orthogonal ($U^T U = I$):

$$X^T X = VD^2 V^T$$

When to use SVD over EVD?

EVD computes $\Sigma = X^T X$, which squares the condition number of the data.

Scenario: Speech Audio
If you have $p = 5000$ audio features and $n = 100$ patients:

- $X^T X$ is a massive 5000×5000 matrix.
- Multiplying $X^T X$ causes severe floating-point precision loss.

Verdict:
Always use SVD directly on X for high-dimensional ($p > n$) biosignal data. It is numerically stable and computationally faster!

Scores and Loadings

The Transformation:

$$S = XL$$

Loadings (L): The Principal Axes

- Size: $p \times m$, where $m = \min(n, p)$
- A rotation matrix: $L^T L = I$
- Columns are orthogonal and of unit length.

Scores (S): The Principal Components

- Size: $n \times m$, where $m = \min(n, p)$
- Coordinates of the data points on the new axes.
- PCs are uncorrelated: $S^T S$ is a diagonal matrix.

Intuition: Audio Features Example

Let X represent patients (n) and audio frequency features (p).

What is a Loading (L)?
It acts as a 'recipe'. A loading vector might say: *'To build PC1, take 80% of frequency A and 20% of frequency B.'* It dictates the direction of the new axes.

What is a Score (S)?
It acts as a 'patient summary'. A score tells you exactly where a specific patient sits along those new PC axes.

Principal components are uncorrelated

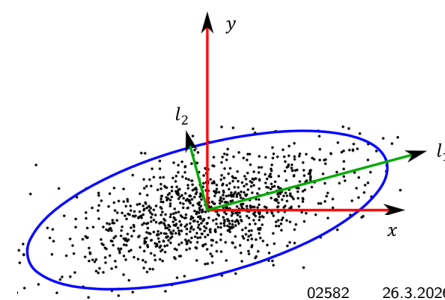
With Gaussian data we have

$$x_i \in N(\mu, \Sigma)$$

and transformed data is

$$s_i \in N(0, D) \quad \text{with} \quad D = \begin{bmatrix} \sigma_1^2 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \sigma_k^2 \end{bmatrix}$$

where $\sigma_1^2 \geq \dots \geq \sigma_k^2$



Dimensionality reduction

The first scores explain most of the variation in data. We can use this to "compress data". Here is data matrix with 4 variables where we keep 3 principal components,

$$[s_1\ s_2\ s_3]_{n\times 3} = [x_1\ x_2\ x_3\ x_4]_{n\times 4}[l_1\ l_2\ l_3]_{p\times 3}$$

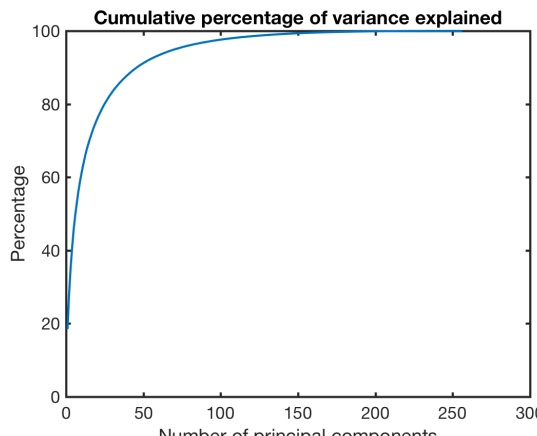
We reconstruct the observations with

$$[x_1\ x_2\ x_3\ x_4] \approx [s_1\ s_2\ s_3][l_1\ l_2\ l_3]^T$$

the error we make equals the omitted term $s_4 l_4^T$ which usually has a lower variance than previous terms.

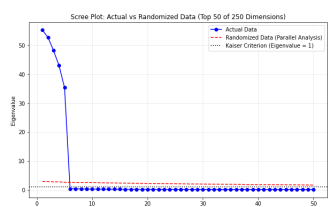
Explained variance

The fewer PCs we keep the less of the variation in data is retained. Example: In the zip code data, the ten first PCs cover 61 % of the variance, and the first fifty PCs cover 91 % of the variance.



The number of components?

- Obtain a certain percentage of explained variance
  - Usually a bad idea
- Keep eigenvectors with eigenvalues greater than one for correlation matrix (Kaiser criterion) - The Kaiser criterion ensures that every latent component you keep explains more information than a single original feature!
- Make a **scree plot** - compare eigenvalues to those obtained from randomized data (with same total variance).



PCA - Computation

1. Eigen analysis of covariance matrix

```
[L D] = eig(cov(X));
```

L - loading matrix
D - diagonal matrix of variances
$\Sigma = LDL^T$

2. Singular value decomposition of data matrix

```
[n p] = size(X);
X = X - ones(n,1)*mean(X);
[U S_d L] = svd(X,'econ');  %X = U*S_d*L'
D = diag(diag(S_d).^2)/n);
```

$\Sigma = \frac{1}{n} X^T X = L \left(\frac{1}{n} S_d^2 \right) L^T$

3. `[loading, score, variance] = pca(X);`

PCA captures main variation

PCA captures main variations in first principal components. This does not always mean that it captures what we are interested in

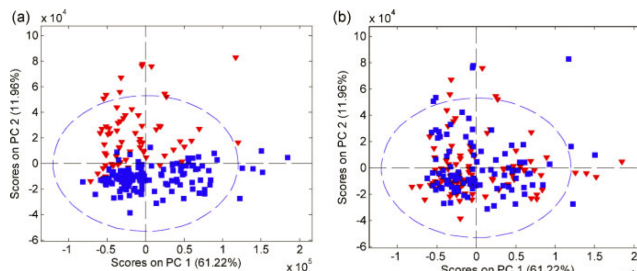


Figure 3. Two identical score plots (PC1, PC2) colored according to two different nonincluded variables: (a) male/female and (b) ill/healthy. Explained variance = 73%.

Kjeldahl & Bro, J. Chemometrics 2010; 24:558-564

Explained variance

Percentage of explained variance is relative and can not be compared

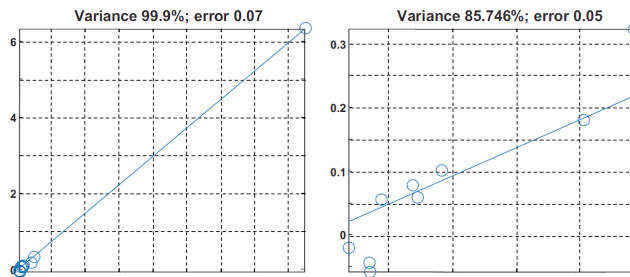


Figure 2. Example of the noncomparability of percentages across different datasets. To the left, a univariate regression describes almost 100% of the variation giving an error of 0.07. Upon removing the extreme upper-right sample, the explained variance is reduced, but so is the error.

Kjeldahl & Bro, J. Chemometrics 2010; 24:558-564

Interpreting loadings

Loadings change with scaling of variables. Use standardized data for interpretation.
Variables close to each other far away from origin (0,0) are correlated.
Variables close to (0,0) are not explained by the chosen principal components and not much can be concluded.

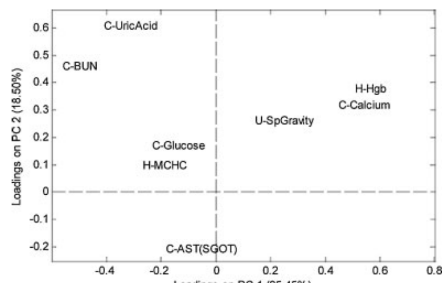


Figure 5. PCA loadings plot. H-Hgb and C-Calcium are highly correlated within the variation explained by PC1 and PC2, whereas nothing similar is certain about C-Glucose and H-MCHC regardless of their close position.

Kjeldahl & Bro, J. Chemometrics 2010; 24:558-564

Scatter plots

Scatter plots of scores might be used for spotting clusters and outliers. Make sure that the scaling on the axes are comparable!

Figure 6. Two maps showing the same part of the world, either true to the surface distances (left), or with different scaling on the axes (right).

Kjeldahl & Bro, J. Chemometrics 2010; 24:558-564

Always

- Causality is not the same as an observed correlation
  - The correlation is only an observed correlation as the relation between sales of ice cream and the number of drowning accidents, where an underlying factor (the sun) is the direct causal link.
- Keep the aim of the modeling in focus
- Support choices by domain specific knowledge
- Be critical

PCA Benefits & Properties

1. Uniqueness & Independence

- The **only** linear transform with mathematically independent loading vectors ($L^T L = I$) and uncorrelated scores ($S^T S$ is diagonal).

2. Optimality in Compression

- Yields the most compact linear data representation possible (minimizes reconstruction error).

3. Computational Efficiency

- Easy and quick to calculate using SVD.
- Solves multicollinearity natively.
- Handles $p > n$ environments seamlessly.

Example: Downstream Data Tasks

The $p > n$ Advantage:
If we have 100 patients but $50,000$ genomic or audio sequences, regression algorithms often crash. SVD computes the PCA in seconds.

Data Pipeline Storage:
We can compress high-dimensional biosignals down to just 3 or 4 Principal Components, retain 95% of the variance, and use these independent scores in a neural network without memory issues.

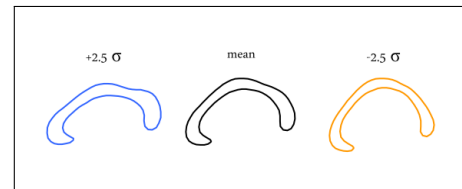
PCA Drawbacks: The Interpretability Crisis

Unsupervised Nature

- PCA maximizes internal variance, which might have zero correlation with the clinical outcome y.

*This lack of interpretability leads us to explore **Sparse PCA** and **PLS**.*

Example: Clinical Explanation



If PC1 represents a global shape change in a biological structure (e.g., Corpus Callosum) combined with local data artifacts, **what is the clinical explanation?**

Doctors need localized, isolated variables (e.g., "width of region A"), not global mathematical composites that smear the entire dataset into one score.

Sparse PCA: The Goal

The Objective:

- We have learned about sparse regression methods (like LASSO).
- We want to apply this to PCA: Approximate s_1 but **drive some coefficients to zero**.

The Spectrum of Sparsity:

- When no sparsity is imposed $\implies$ **Regular PCA**.
- When maximal sparsity is imposed $\implies$ $L = 0$ (Useless).
- **Sparse PCA** aims for the sweet spot: Maximize variance, keep independent loadings, but force irrelevant variable weights to exactly zero.

Visualizing the Goal

Regular PC1 Loading Vector:
$L_1 = [0.1, \quad 0.8, \quad -0.3, \quad 0.4, \quad 0.2]^T$
(Dense: Uses every variable)

Sparse PC1 Loading Vector:
$L_1 = [0.0, \quad \mathbf{0.9}, \quad 0.0, \quad \mathbf{0.4}, \quad 0.0]^T$
(Sparse: Isolates only the two most important variables)

How do we calculate this? 1. Thresholding of loadings, 2. Varimax rotation of loadings, 3. estimation using Elastic Net.

Sparse PCA

- When no sparsity is imposed, we would like to have the regular PCA.
- For maximal sparsity, $L = 0$

Sparse PCA: The Thresholding Trap

Method 1: Thresholding

- *Idea:* Simple! Set all loadings below some arbitrary threshold (e.g., $|l_j| < 0.15$) to zero.

Why is this a bad idea?

- Think of **Ridge Regression**.
- Coefficients in a linear combination are **interdependent**.

$$s_1 = l_{11}x_1 + l_{21}x_2 + l_{31}x_3 + l_{41}x_4$$

- If you artificially zero out l_{31}, the remaining coefficients l_{11}, l_{21}, l_{41} are no longer mathematically optimal.

Example: Breaking the Subspace

When you arbitrarily threshold weights:

1. Orthogonality is Lost:
$L^T L \neq I$. The principal axes are no longer perpendicular.

2. Scores are Correlated:
$S^T S$ is no longer diagonal. The features you extract now contain redundant information.

Conclusion: Thresholding provides visual sparsity, but destroys the mathematical guarantees of PCA.

Rotated Principal Components

The varimax criterion

- Produces approximately sparse loading vectors
- Quick to compute
  - ...but we skip the algorithm here
- Concept,

$$S = XL \iff X = SL^T$$
$$X = SR^T RL^T = (SR^T)(LR^T)^T$$

$$\tilde{S} = SR^T \text{ rotated } S$$
$$\tilde{L} = LR^T \text{ rotated } L$$

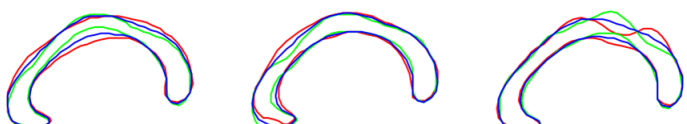
R rotates in $\mathbb{R}^p \implies L$ stays orthogonal, S does not.

The Varimax Criterion

Maximize the variance among the loadings in each principal axis gives approximate sparseness.

- Some loadings are high, some are close to zero.
- Amount of sparsity is determined by k, the number of retained PCs.

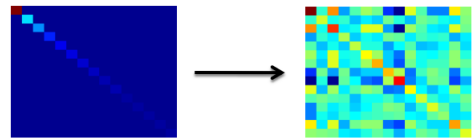
Example,



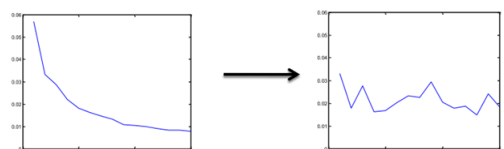
1st mode 2nd mode 3rd mode

The Varimax Criterion

- Scores become correlated, $S^T S$,



- Eigenvalue spectrum becomes flattened



SPCA using the elastic net

Express each PC as a regression problem

$$\arg\min_{l} ||x_i - Xl||^2$$

Problem: Cannot be solved when $p > n$.
Solution: Ridge regression

$$\arg\min_{l} ||x_i - Xl||^2 + \lambda ||l||^2$$

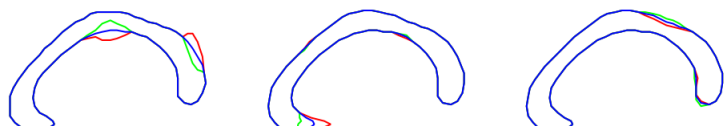
- Normalize solution to unit length
- Result: Standard PCA. Works well when $p > n$

SPCA using the elastic net

Add a L_1-penalty to get sparse solutions

$$\arg\min_{l} ||x_i - Xl||^2 + \lambda||l||^2 + \gamma||l||_1$$

Example,



Drawback, solution is guided by the original principal components.

Principal Component Regression, PCR

PCR: PCA + Linear regression

$\rightarrow$ Linear regression on the PCA scores

From a PCA analysis of data X we have the scores S. Use $[s_1,\ s_2 \ldots, s_M]$ for some $M \leq p$ and we have a standard regression problem in the new variables,

$$y = \beta_0 + [s_1, \ldots, s_M]\beta + e$$

- PCR handles $n < p$ by operating on a subset of PCs.
- PCR performs similar to ridge regression
- Equivalent to OLS when $M = p$

Partial Least Squares

- Supervised method with latent variable structure
- Seeks directions which have high variance and have high correlation with the response
- Tune number of PLS components

Partial Least Squares

The mth PLS direction φ_m solves

$$\max_{\alpha} Corr^2(y, X\alpha)Var(X\alpha)$$

$$\text{subject to } ||\alpha|| = 1,\ \alpha^T \Sigma \varphi_l = 0,\ l = 1, ..., m-1$$

- Behaves similar to ridge regression and principal component regression in shrinking coefficient estimates.
- Shrink low variance directions (like ridge)
- Can inflate high variance directions

Partial Least Squares

- Many algorrithms exist
- A linear regression in X, with a latent structure explaining the most of the y variation
- Deals with multicolinearity
- Can handle several response variables

Partial Least Squares (PLS): Supervised Subspaces

Unsupervised vs. Supervised:

PCA: Seeks v to maximize internal variance.

$$\max_{v} \mathsf{Var}(Xv)$$

PLS: Seeks u, v to maximize covariance with a target y.

$$\max_{u,v} \mathsf{Cov}(Xu, Yv)$$

$\mathsf{Cov}(Xu, Yv)^2 = \mathsf{Var}(Xu)\mathsf{Var}(Yv)\mathsf{Corr}^2(Xu, Yv)$. PLS explicitly balances variance with correlation.

Example: Wrist Angel (OCD)

Let X be wearable biosignals (Heart Rate, Actigraphy) and y be OCD severity.

The highest variance in X (PCA) might just be the child running or playing sports. PLS ignores this movement noise and finds the hidden subspace of physiological signals that directly correlate with an OCD episode.

The Danger of PCR vs. The PLS Solution

Principal Component Regression (PCR):

1. Run PCA on X to get scores S_k.
2. Run OLS regression: $\hat{y} = S_k\beta$.

The Flaw: The directions of highest variance in X might have **zero** correlation with y. PCA might discard the most predictive features!

The PLS Objective (The Fix):

The m-th PLS direction α_m solves:

$$\max_{\alpha} \text{Corr}^2(y, X\alpha)\text{Var}(X\alpha)$$

s.t. $\|\alpha\| = 1$ and is orthogonal to previous components.

Example: Audio Emotion

In a Speech Emotion task ($y =$ anger level):

PCR Failure: PCA keeps the loud background hum (high variance) but drops a subtle, high-frequency vocal quiver (low variance). PCR fails to predict anger.

PLS Success: PLS forces the subspace to look for features that explain y. It elevates the subtle quiver because $\text{Corr}^2(y, X\alpha)$ is very high.

Canonical correlation analysis

Finds associations between two data groups.

The canonical correlation of the data matrices X and Y are given as,

$$\max_{u_m, v_m} Corr^2(Yu_m, Xv_m)$$

subject to $u_m u_j = 0$ and $v_m v_j = 0 \; m \neq j, m = 1, ..., M$

The linear combinations are uncorrelated and there is at most the minimum dimension of the two data matrices.

Canonical Correlation Analysis (CCA): Multi-Modal

Aligning Two Views:
We have two distinct views (X and Y) of the same subject. We seek canonical variates $U = Xu$ and $V = Yv$.

The Objective:

$$\max_{u,v} \frac{u^T \Sigma_{XY} v}{\sqrt{u^T \Sigma_{XX} u \cdot v^T \Sigma_{YY} v}}$$

Unlike PLS, CCA ignores internal variance; it focuses entirely on maximizing the cross-correlation between domains.

Example: Clinical Alignment

View X: Vocal features extracted from patient clinical interviews.
View Y: Self-reported psychological questionnaire scores (e.g., Anxiety, Sleep Quality).

CCA automatically finds the mathematical combination of vocal features that most strongly aligns with the patients' reported psychological states.

CCA Derivation: Lagrangian & Constraints

The Lagrangian:
Maximize covariance subject to unit variance constraints: $u^T \Sigma_{XX} u = 1$ and $v^T \Sigma_{YY} v = 1$.

$$\begin{aligned}\mathcal{L}(u, v, \lambda_1, \lambda_2) = & u^T \Sigma_{XY} v \\ & - \frac{\lambda_1}{2}(u^T \Sigma_{XX} u - 1) \\ & - \frac{\lambda_2}{2}(v^T \Sigma_{YY} v - 1)\end{aligned}$$

Example: Why Constrain Variance?

If we simply told the math to 'maximize covariance' without the constraints ($= 1$), the algorithm would just scale the weights (u and v) to infinity to make the score larger.

Constraining the projected variance forces the model to focus purely on the *structural relationship* between the Audio features and the Questionnaires, not the magnitude of the numbers.

CCA: The Generalized Eigenvalue Problem

The Solution:
Taking partial derivatives and substituting yields a Generalized Eigenvalue Problem:

$$\Sigma_{XY}\Sigma_{YY}^{-1}\Sigma_{YX}u = \lambda^2\Sigma_{XX}u$$

- u: Canonical weights (eigenvectors)
- λ: Canonical correlation (sqrt of eigenvalues)

Example: Python Implementation

This math maps directly to code! In our Python demo, calculating this eigenvalue problem manually:

```
M = inv(S_XX) @ S_XY @ inv(S_YY) @ S_YX
eig_vals, eig_vecs = np.linalg.eig(M)
```

Produces the exact same canonical weights as calling `sklearn.cross_decomposition.CCA`.

Sparse CCA

- Several regularized and sparse versions of CCA exist
- Ridge regularization: Regularized CCA (Vinod 1976, Leurgans et al 1993)
- Sparse version: Penalized Matrix Decomposition (Witten et al, 2009)

High-Dimensional CCA: Sparse / Regularized

The Computational Catch:
The math requires inverting Σ_{XX} and Σ_{YY}.

Regularized CCA (Ridge):

$$(\Sigma_{XX} + \lambda_X I)^{-1}\Sigma_{XY} \dots$$

Sparse CCA (PMD):
Applies L_1 penalties to u and v to yield sparse canonical vectors.

Example: The $p \gg n$ Failure

If you extract 3000 audio features from 50 patients, the covariance matrix Σ_{XX} is **singular** (non-invertible).

Standard CCA mathematically crashes. Sparse CCA solves the invertibility problem while simultaneously zeroing out unhelpful audio features, leaving only the biomarkers.

Exercises

- Shape analysis of faces
- PCA inside-and-out
- Sparse PCA
  - Thresholding
  - Varimax rotation
  - Simple Elastic Net version
- PLS on sand data

Exercise - Shape analysis of faces

1. Apply Principal Component Analysis (PCA) to the face data set in *faces.mat*
   a. Load the face shape data in the file *faces.mat*
   b. Compute the mean shape and center the data. Plot the mean shape (see details in exercise file)
   c. Compute a principal component analysis of the data. Try using both an eigen value decomposition (EVD) and a singular value decomposition (SVD). Remember that the EVD is computed on the correlation or covariance matrix and the SVD on the data matrix itself. See calculation hints in exercise file.

Exercise 1 - Shape analysis of faces

PCA continued - illustrations

d Plot the first mode of variation. This will provide a view of the most important variation in the data set. Let the mean face be the origin, we will use the mean face as a reference. Plot the mean face in black.

- Compute the face obtained by moving from the mean face along the first principal axis (first column of the loading matrix) out to a distance of $+2.5$ standard deviations ($\mu + 2.5\sigma_{l_1}$). The standard deviations can be obtained from the singular values (see the slides on how this is done). Plot the resulting face in red.
- Repeat this procedure to obtain a face at -2.5 standard deviations from the mean face. Plot this in blue.

d Explore the first few modes of variation using the provided shape inspector.

Exercise 2 - Sparse PCA

Extract Sparse Principal Components for the face data using **three different methods**:

a. Compute a sparse PCA by **thresholding** all loadings from a regular PCA with absolute value less than 0.15.
   - Now that the loading matrix has been changed, the scores matrix must be recomputed. How can you use this new scores matrix to compute the variance of the data along each principal axis?
   - Use this result to plot the first mode of variation of the threshold SPCA for -2.5, 0 and +2.5 standard deviations.
   - Investigate what has happened with the uncorrelatedness of the loadings and scores matrices of regular PCA. Are these properties fulfilled here?

b. Use the **Varimax criterion**. Experiment with the number of columns from the loading matrix to rotate, and see how this affects sparsity.
   - Again, what has happened with the uncorrelatedness of the scores and loading matrices? See the code listings for functions which perform Varimax rotation.

c. Compute the first sparse principal loading vector using the **Elastic net** (remember to normalize to unit length).
   - Start by using 10 non-zero loadings. Plot this solution and try different number of nonzero components.
   - Can you put a meaningful anatomical label on this deformation?
   - Would you be able to label the first mode of variation from regular PCA?

Exercise 3 - PLS on sand data

Apply Partial Least Squares regression to the sand data.

a. Load data *sand.mat* and run a cross validation of partial least squares regression to decide the number of components that is adequate to model the sand data. Plot both the cross validation error and the percentage of explained variance in y to determine the number of components. See the code listings for useful PLS implementations.

b. How would you plot the coefficients of the final PLS regression model (β)? Which variables are important for the prediction of y? (In terms of loadings, remember the pitfall of PCA concerning scaled vs non-scaled loadings - this also holds for PLS).

PLS Algorithm: Iterative

Partial Least Squares Algorithm

- Standardize the columns of X, $\hat{y}^{(0)} = \mu_y$, and $x_j^{(0)} = x_j$ for $j = 1, ..., p$.
- For $m = 1 : M$
  - $z_m = \sum_{j=1}^{p} \hat{\varphi}_{mj} x_j^{(m-1)}$, where $\hat{\varphi}_{mj} = x_j^{(m-1)^T} y$
  - $\hat{\theta}_m = z_m^T y / z_m^T z_m$
  - $\hat{y}^{(m)} = \hat{y}^{(m-1)} + \hat{\theta}_m z_m$
  - Orthogonalize each $x_j^{(m-1)}$ w.r.t z_m, $j = 1, ..., M$
  - Output: $\hat{y}^{(m)}$ and $\hat{\beta}_j^{PLS} = f(\varphi_{.j})$.
- Equivalent with OLS when $M = p$
- The components z_i and z_j are uncorrelated $(i \neq j)$

Toy Numerical Example ($m = 1$)

Let $X = [x_1, x_2]$ and y be standardized.
Suppose $x_1 = [1, -1]^T$, $x_2 = [1, 1]^T$, $y = [2, -2]^T$.

1. Weights (Covariance with y):
$\hat{\varphi}_{11} = x_1^T y = (1)(2) + (-1)(-2) = 4$
$\hat{\varphi}_{12} = x_2^T y = (1)(2) + (1)(-2) = 0$

2. Latent Component z_1:
$z_1 = 4x_1 + 0x_2 = [4, -4]^T$
Insight: PLS automatically ignored x_2 because it had zero correlation with y!

3. Predict y from z_1:
$\hat{\theta}_1 = z_1^T y / z_1^T z_1 = 16/32 = 0.5$
$\hat{y}^{(1)} = \hat{y}^{(0)} + 0.5 z_1 = [2, -2]^T$ (Perfect fit)

Variable Definitions

Before detailing the algorithm, let's define the key variables used in the iterative process:

- $X \in \mathbb{R}^{n \times p}$: Standardized feature matrix (n samples, p features).
- $y \in \mathbb{R}^n$: Standardized target vector.
- M: Desired number of latent components to extract ($M \leq p$).
- m: Current iteration/component counter ($m = 1, \ldots, M$).
- $x_j^{(m-1)}$: The j-th feature column vector after $m-1$ deflations.
- $\hat{\varphi}_{mj}$: Weight (covariance) of feature j for component m.
- $z_m \in \mathbb{R}^n$: The m-th extracted latent component (score vector).
- $\hat{\theta}_m$: The scalar regression coefficient for component m.
- $\hat{y}^{(m)}$: The prediction vector at iteration m.

Step 0: Initialization

Before extracting components, we must prepare the data to ensure they are on the same scale and centered at zero.

1. **Standardize** the columns of the feature matrix X (mean 0, variance 1).
2. **Standardize** the target vector y (mean 0, variance 1).
3. **Initialize Predictions:**
$$\hat{y}^{(0)} = \mu_y = \mathbf{0}$$
(Since y is standardized, its mean is a zero vector).
4. **Initialize Features:**
$$x_j^{(0)} = x_j \quad \text{for } j = 1, \ldots, p$$

The Iterative Loop: Steps 1 & 2

For each component $m = 1 \dots M$:

1. Calculate Weights (Covariance)

Find the correlation/covariance between each updated feature and the target. Features highly correlated with y get larger weights.

$$\hat{\varphi}_{mj} = x_j^{(m-1)^T} y$$

2. Construct Latent Component (z_m)

Combine the features, weighted by their correlation to y.

$$z_m = \sum_{j=1}^{p} \hat{\varphi}_{mj} x_j^{(m-1)}$$

The Iterative Loop: Steps 3 & 4

Still inside the loop for component m:

3. Calculate Regression Coefficient ($\hat{\theta}_m$)

Find the coefficient for regressing y onto our new latent component z_m using standard Ordinary Least Squares (OLS) logic.

$$\hat{\theta}_m = \frac{z_m^T y}{z_m^T z_m}$$

4. Update Prediction ($\hat{y}^{(m)}$)

Add the contribution of this new component to the running prediction.

$$\hat{y}^{(m)} = \hat{y}^{(m-1)} + \hat{\theta}_m z_m$$

The Iterative Loop: Step 5 (Deflation)

The final step in the loop prepares the data for the next component $m+1$.

5. Orthogonalize (Deflation)

Orthogonalize each feature $x_j^{(m-1)}$ with respect to the component z_m. We subtract the projection of x_j onto z_m.

$$x_j^{(m)} = x_j^{(m-1)} - \left(\frac{z_m^T x_j^{(m-1)}}{z_m^T z_m} \right) z_m \quad \text{for } j = 1 \dots p$$

Insight: This strips away the variance that z_m just explained from our feature matrix, ensuring that the next component z_{m+1} will capture entirely new information.

Key Properties of PLS

Upon completing the loop, the algorithm yields the final prediction $\hat{y}^{(M)}$ and a set of latent components $\{z_1, \ldots, z_M\}$.

Mathematical Guarantees:

- **Uncorrelated Components:** By design of the orthogonalization (deflation) step, the extracted latent components are orthogonal to each other.

$$z_i^T z_j = 0 \quad \text{for all } i \neq j$$

- **OLS Equivalence:** If we choose to keep all possible components ($M = p$), no dimensionality reduction occurs, and the PLS predictions are mathematically equivalent to Ordinary Least Squares (OLS) regression.